



15.4 Student Guide: Post-Exploitation and Reporting

Overview

Today, you will learn more about the post-exploitation phase and common post-exploitation tasks, followed by reporting. You will learn about reperforming enumeration after privilege escalation and how to gather password hashes on the machine and crack them offline with John the Ripper. You will then learn about establishing persistence by creating a backdoor account. Finally, you will review reporting and explore a report template that you'll use for the deliverable at the end of the next module.

Class Objectives

By the end of class, you will be able to:

- Describe the common tasks that are included in privileged post-exploitation.
- Perform post-exploitation tasks, such as gathering password hashes.
- Explain how password crackers work and perform password cracking.
- Understand the importance of reporting and fill out a strong report.

01. Post-Exploitation Tasks

In this lesson, we will continue exploring the **post-exploitation** stage of a pen test engagement and then cover the last stage, **Reporting**.

Before beginning, let's review the important concepts covered on Day 3:

- **Command and control (C2)** is a framework that consists of tools and techniques that attackers use to maintain communication with compromised devices following initial exploitation.
- The C2 architecture consists of:
 1. **C2 server:** The attacker's server, where the attacker communicates with the compromised machines.
 2. **C2's agents:** The payload that is run on the compromised machine to open a connection back to the C2 server

- Once the machine has the C2's agent running on it, it is compromised and sometimes referred to as a **zombie**.
 - Multiple zombies form a **botnet**.
- **Metasploit** is a popular C2 framework that contains a suite of tools for enumerating and exploiting servers and other networked devices. The main tools of Metasploit are:
 - **MSFconsole**: The main interface for Metasploit, which runs on your local machine
 - **Meterpreter**: A Linux-style shell that runs on the machines you compromise
 - **msfvenom**: Allows the operator to craft the malicious agents and payloads that will be used to communicate back with Metasploit
- MSFconsole is a unified interface for a variety of functions. Each of these functions is called a **module**.
- The five main types of Metasploit modules are:
 - **Auxiliary modules**: Used for information gathering, enumeration, and port scanning
 - **Exploit modules**: Generally used to deliver exploit code to a target system
 - **Post modules**: Offers post-exploitation tools
 - **Payload modules**: Used to create malicious payloads to use with an exploit
 - **Encoder modules**: Used to ensure that payloads make it to their destination intact
- **Post-exploitation** is the next phase and involves any actions performed *after exploitation*. It includes three common tasks:
 - Enumeration and searching for useful data: This is similar to the enumeration process during the Reconnaissance phase; however, we are enumerating and searching for data from inside the target after it has been exploited.
 - Persistence: This is the method of attempting to maintain long-term access to your target.
 - Privilege escalation: This is the method of escalating privileges from a low-privilege to high-privilege user.

Reperforming Enumeration After Privilege Escalation

- We finished the previous lesson by performing the post-exploitation task of **enumeration** and utilizing that data to conduct **privilege escalation**.
 - **Low-privileged users** have limited access to the machine. For Linux, this typically means not being able to browse through configuration files or other home folders.
 - **High-privileged** users have more access. In the case of root, privileges are unrestricted.

Since we are now in a high-privileged or even root context, we now have more post-exploitation tasks to perform since we have more access to the file system. We can now **reperform enumeration** as a high-privilege user to see if there are any other useful files.

On Linux, several files would now be accessible and could offer additional information to an attacker, such as:

- `/var/lib/mysql/mysql/user.MYD` is a MySQL database file, which may contain SQL login credentials.
- `/home/[USER]/.bash_history` contains a history of a user's bash commands.
- `/home/[USER]/.ssh/` contains private SSH keys.

Can you remember another file in Linux that is extremely sensitive and would be fruitful for an attacker? The `/etc/shadow` file is one of the most sensitive files on Linux, because it houses the password hashes for all users on the machine. This file is typically limited to root-level access.

While the `/etc/shadow` file appears nearly identical to `/etc/passwd`, there is one stark difference. Note this with the following record for the user, **timmy**, in each file:

- `/etc/passwd: timmy:x:1016:1019::/home/timmy:/bin/sh`
- `/etc/shadow:`
`timmy:$6$6Y/fI1nx$zQJj6AH9asTNfhxV7NoVgxByJyE.rVKK6tKXi0GNCfWBsrTGY7wtC6Cep6co9eVNkRFrpK6koXs1NU3AZQF8v/`

Instead of an `x`, like in the `passwd` file, the second field in the `shadow` file (after the `:`) contains a long, complex string:

`$6$6Y/fI1nx$zQJj6AH9asTNfhxV7NoVgxByJyE.rVKK6tKXi0GNCfWBsrTGY7wtC6Cep6co9eVNkRFrpK6koXs1NU3AZQF8v/`.

Note the following about this field in the `shadow` file: The string is a password hash. This hash cannot be used to log in, but can be cracked to retrieve the user's original password. If a row in `/etc/shadow` contains an `!` instead of a password hash, it means the account is locked. If a row in `/etc/shadow` contains an `*` instead of a password hash, it means the user is not allowed to log in. This is usually implemented on a user reserved for the system, and not a human user.

Now that we have `sudo` or root access, we can read this file and obtain the hashes, which we can crack offline. We want to crack additional passwords to gain access to other accounts, which may have additional privileges across the network.

In the next demonstration, we will continue our post-exploitation activities by revisiting **John the Ripper** to crack the hashes from the shadow file.

Password Cracking Refresher

How to use `john`:

1. Copy the following hash into a file called `hash.txt` with the following command:
 - `echo '$6$RhLGADK.vMFF4Gzf$G.3Y5tHVGcJ096M3gwRi1TkyxLSNbr4QJEDHQhdjUBwJX2Hf05RUAvgnHOWD4FkVEsiuc5fQGps4MegSbPD36.' > hash.txt`
2. Use `john` with the file as an argument to crack the hash.
 - `john hash.txt`
3. Note that the hash was cracked to reveal that the password is "password".
 - Note that `john` will automatically detect the hash type.
4. Note that `john` uses a default wordlist but can use other wordlists as well.
 - Show an example of using a wordlist called `passwords.list`:
 - `john --wordlist=passwords.list hash.txt`
5. If the password doesn't show once it's cracked, type
 - `john --show hash.txt`

In the next activity, you will use John the Ripper to conduct a post-exploitation password-cracking activity.

02. Activity: Password Cracking

- [Activity File: Password Cracking](#)

03. Persistence

Now that we have completed the post-exploitation task of enumerating data after privilege escalation, we'll move on to another post-exploitation task: establishing **persistence**.

- Per MITRE, **persistence** consists of techniques that adversaries use to keep access to systems across restarts, changed credentials, and other interruptions that could cut off their access. Persistence techniques include any access, action, or configuration changes that allow attackers to maintain their foothold on systems, such as replacing or hijacking legitimate code or adding startup code.
- If a burglar broke into a building and wanted to return later without getting caught, they could unlock a window in the back of the building so they could sneak back in.

- Persistence is its own tactic under MITRE: <https://attack.mitre.org/tactics/TA0003/>.

Persistence Techniques

"Persistence" refers to an adversary maintaining their access to the target, but there are several techniques that can be used to maintain persistence.

Note that often, several of these techniques may be applied together.

- **Technique 1: Utilizing Existing Services**

- Persistence is less about the exploitation of vulnerable software and services and more about utilizing existing system services.
 - For example, utilizing `useradd` to create a new user as a "backdoor."
- Can you think of any additional ideas about how to use existing system services or functions for persistence?
 - Creating a cron job to schedule the execution of a payload/reverse shell that provides access
 - Creating an additional user to SSH
 - Creating a script that executes a reverse shell when a user logs on
 - Uploading a web shell page to the existing webpage directory (usually `/var/www/html`)

- **Technique 2: Blending into the Environment**

- An important part of persistence is making sure that the attacker's work blends in with the current environment and can't be detected.
- Knowledge of the system you're on is crucial to blending in with the environment so as not to be caught.
 - For example, if creating a backdoor account, you want to blend in with current account names instead of naming the account `backdoor`. (E.g., Use `sysadmin` to pose as a system admin or `systemd-ssh` to pose as a service.)

- **Technique 3: Maintaining Current Level of Access**

- Another key aspect of persistence is maintaining your current level of access.

- You could, and typically should, establish persistence as a low-privilege user before privilege escalation, just in case you lose access to the machine.
 - You should then reestablish persistence as an elevated user.
- Note that elevated users will have more options for persistence due to privileged access to service e.g., crontab.
- It's important to establish and ensure persistence with the highest privileges possible to avoid having to re-step through the privilege-escalation phase.

- **Technique 4 - Privesc Persistence**

- There are also opportunities for **privesc persistence**, which means purposely establishing opportunities for low-privilege users to escalate their privileges.
 - For example, the SUID bit technique: <https://attack.mitre.org/techniques/T1548/001/>.
 - On Linux or macOS, when the setuid or setgid bits are set for an application, the application will run with the privileges of the owning user or group.
 - Normally, an application is run in the current user's context, regardless of which user or group owns the application. However, sometimes programs need to be run in an elevated context to function properly, but the user running them doesn't need the elevated privileges.
- As an attacker with root privileges, you could establish a privesc persistence script by creating a reverse shell in a Python script and setting the SUID bit to root.

Summary

- In the post-exploitation stage, after privilege escalation, we can now **reperform enumeration**.
 - This is because we now have access to higher privileged files, such as the shadow file.
- **Persistence** is another post-exploitation method, that an attacker can utilize to maintain access to their target.
 - Techniques that can be used for persistence include:
 - **Utilizing existing services**
 - **Blending into the environment**
 - **Maintaining the current level of access**
 - **Privesc persistence**

Ultimately, we use persistence to get back into the target and conduct further exploits.

04. Activity: Persistence

- [Activity File: Persistence](#)
-

05. Break

06. Reporting

While we have completed all the technical aspects of our penetration test, we have one important step left.

We will complete the penetration test engagement by completing the most important phase, **Reporting**.

- **Reporting** is the most important phase because it's where the pen tester describes their actions in words, which will eventually be communicated to stakeholders and leadership who will interpret the results.
- To the client, the quality of the report reflects the quality of the pen tester's work.
 - A poorly written report could make the client believe the pen tester is incompetent and the pen test was a failure, even if the pen tester had several important findings.
 - Communication is the key component in linking actions to results.
 - A well-written report will help establish the pen tester's reputation and perhaps generate repeat business, while also taking a positive step toward remediating any findings within the client's network.

Report Writing Best Practices

Note the following tips for creating a high-quality report:

- The report should have a good balance between technical and non-technical results.
- The findings should be understandable by management, who are usually less technical, but there should also be a more technical section that system administrators can use to develop remediation strategies.
- It's extremely important to document and log all actions taken, even if they were unsuccessful. Sometimes engagements trigger alerts, and the pen tester needs to be able to prove with timestamps that it was their actions that caused the alert to fire and not an actual adversary.
- The report should also contain any notable failed actions that the pen tester attempted. For example, if anti-virus software successfully stopped a payload from executing, that should be documented in the report. It shows that the current defensive stack for the customer is effective, which is a form of positive reinforcement.

Reporting Overview

In the upcoming activity, you will fill out part of the report template. We will finish the rest after the next module.

Open [this report](#).

- Fill out anything that's highlighted.
- Use the notes that you kept from the previous activities to complete the report.

Page 1. This is the cover/title page of the report. Come up with a company name and fill out your name and the date on the title page.

Page 4. Contact information should be put here. You do not need to use your real details and qualifications for this exercise. In the **document history** table, keep track of any revisions made to the report.

Page 5. The introduction sets the stage by explaining why the engagement was undertaken and the expected deliverables. You are only responsible for completing the highlighted items.

Pages 8–10. The executive summary provides a quick snapshot of what occurred during the engagement. This section is meant to be read quickly by high-level executives or stakeholders, so keep technical content to a minimum.

Page 8. Grading methodology serves as a rubric for scoring any findings. These should not be changed. Organizations use these gradings to prioritize the items they need to fix.

- This page is for logging actions taken to distinguish the pen tester's actions from another, potentially real, attack.

Page 10. The executive narrative tells the story of the steps that the pen tester took during the engagement. Avoid adding too much detail, but using technical terms is OK here.

Pages 11–12. The main vulnerability findings should go in this section.

- Provide your best estimate for the severity of each finding.
 - To guide your severity ratings, you should consider what the impact on the company would be if the vulnerability were exploited.
- Under each finding, you can feel free to detail steps taken to reproduce it, as well as any images.

Page 13. Fill out the MITRE map to map the pen tester's actions to MITRE. This serves as a great visual aid, especially for executives who like a visual snapshot of what actions were

successful and unsuccessful during the engagement. When you fill out the MITRE map, save the file, as pointed out in the sample report.

In the next activity, you will complete this report with the findings that you've gathered during penetration testing week.

07. Activity: Reporting

- [Activity File: Reporting](#)